This actor allows you to `ReserveFunds` for a bet. When a bet is created, it is necessary to check if the required funds are available. Like all others in the actor model, this message is processed individually and provides the consistency you need. You know there are no race conditions when funds are requested. Funds are granted only if they are available when the message is processed. If that is the case, a `FundsReserved` event is generated; otherwise, a `FundsReservationDenied` event is generated. You can also use `AddFunds`, which generates a `FundsAdded` event. And there's `CheckFunds`, which does exactly what it says. These events are shown in the following snippet:

```
object Wallet {
  ...

  sealed trait Command extends CborSerializable
  case class ReserveFunds(
      amount: Int,
      replyTo: ActorRef[UpdatedResponse])
      extends Command                                                    ❶
  case class AddFunds(amount: Int, replyTo: ActorRef[UpdatedResponse])   ❶
      extends Command                                                    ❶
  case class CheckFunds(replyTo: ActorRef[Response]) extends Command     ❶

  sealed trait Event extends CborSerializable
  case class FundsReserved(amount: Int) extends Event                    ❷
  case class FundsAdded(amount: Int) extends Event                       ❷
  case class FundsReservationDenied(amount: Int) extends Event           ❷
  ...
}
```

❶ Commands

❷ Events

There is a `Response` and an `UpdatedResponse` in its protocol. A command can be accepted or rejected or get back the current balance: